Lost in the Middle: How Language Models Use Long Contexts

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, Percy Liang Stanford University • Meta AI

TACL, 2023 • arXiv:2307.03172

https://arxiv.org/abs/2307.03172

Abstract

While recent language models have the ability to take long contexts as input, relatively little is known about how well they use longer context. We analyze the performance of language models on two tasks that require identifying relevant information in their input contexts: multi-document question answering and key-value retrieval. We find that performance can degrade significantly when changing the position of relevant information, indicating that current language models do not robustly make use of information in long input contexts. In particular, we observe that performance is often highest when relevant information occurs at the beginning or end of the input context, and significantly degrades when models must access relevant information in the middle of long contexts, even for explicitly long-context models. Our analysis provides a better understanding of how language models use their input context and provides new evaluation protocols for future long-context language models.

1. Introduction

Recent large language models can process long contexts as input, with context windows ranging from 4K to 128K+ tokens. However, relatively little is known about how well these models actually use information distributed across long contexts. This paper studies this question empirically through two controlled tasks: multi-document question answering and key-value retrieval.

The practical motivation is clear: in RAG systems, multiple retrieved documents are concatenated into the context window. The assumption is that the model will attend to whichever document contains the relevant information, regardless of its position. This paper shows this assumption is largely false.

2. Main Finding: The U-Shaped Performance Curve

The central finding is a U-shaped performance curve: language model accuracy is highest when relevant information appears at the beginning or end of the context, and significantly lower when relevant information appears in the middle.

Position of Relevant Doc	Approx. Accuracy (20-doc context)	Relative Performance
Position 1 (beginning)	~71%	100% (baseline)
Position 5	~57%	-20%
Position 10 (middle)	~53%	-25%

Position 15	~58%	-18%
Position 20 (end)	~65%	-8%

This U-shape was observed across multiple model families (GPT-3.5-Turbo, Claude 1.3, LongChat, MPT) and persisted even for models explicitly designed for long contexts. The effect is more pronounced with more retrieved documents.

3. Why This Happens — Attention Analysis

The U-shape reflects two known biases in transformer attention patterns:

Primacy Effect: Transformer attention gives disproportionate weight to early tokens in the sequence, particularly in autoregressive (causal) attention used by decoder-only models. The first tokens receive gradient signal from all subsequent token predictions, making them better represented in the model's internal state.

Recency Effect: The final tokens before the generation point are most recent in the KV-cache and receive high attention weight due to positional encoding proximity. This is especially pronounced in models using RoPE (Rotary Positional Encoding) where positional distance directly affects attention score.

The middle of the context suffers from neither effect: it is not recent (recency bias misses it) and not early (primacy bias has already been applied). Information in the middle is effectively "forgotten" relative to the endpoints.

4. Scaling Behavior

The study found that performance generally decreases as more documents are retrieved and placed in context, even when the relevant document is included. This is counter-intuitive: adding more context (including relevant information) can hurt performance.

Number of Retrieved Docs	Accuracy (relevant doc at best position)	Accuracy (worst position)
1	~77%	~77%
5	~73%	~63%
10	~71%	~57%
20	~70%	~52%
30	~67%	~49%

This finding has a direct implication for RAG system design: the common practice of retrieving top-20 or top-30 documents can actively hurt performance compared to a well-chosen top-5 with reranking.

5. Key-Value Retrieval Experiments

To isolate the position effect from content understanding, the authors also tested key-value retrieval: given a JSON-like object with many key-value pairs, retrieve the value for a specified key. This requires

no reasoning — only locating the right position. Even in this simple task, performance degraded dramatically for keys in the middle of long lists, confirming the effect is architectural rather than semantic.

6. Implications for RAG System Design

The paper's findings directly inform how to assemble the context window in RAG systems:

Recommendation 1 — Place most relevant chunks first or last: The single most impactful change. Put the highest-scoring chunk at position 1 of the context. Put the second-highest at the last position. Fill the middle with lower-scoring context. This alone can recover 10-20% accuracy over naive ordering.

Recommendation 2 — Fewer, more relevant chunks: top_k=5 with good reranking outperforms top_k=20 with mediocre ranking. The reranker becomes critically important because you want to be selective.

Recommendation 3 — Reader saturation: The paper shows that "LM readers saturate before retriever recall" — meaning even perfect retrieval recall does not improve accuracy beyond a saturation point because the model cannot effectively use all the retrieved context. Retrieval precision matters more than recall at high values.

Recommendation 4 — Query-aware contextualization: Placing the query both at the beginning AND end of the context (surrounding the retrieved documents) helps the model maintain focus on what is being asked.

6.1 Practical Context Assembly Pattern

```
QUERY: {user_query} CONTEXT: [Doc 1 - HIGHEST SCORE]: {chunk_1_text} <- position 1 (primacy) [Doc 2 - SECOND SCORE]: {chunk_2_text} [Doc 3 - THIRD SCORE]: {chunk_3_text} <- middle (will be underutilized) [Doc 4 - FOURTH SCORE]: {chunk_4_text} [Doc 5 - FIFTH SCORE]: {chunk_5_text} <- position N (recency) REMINDER - Answer the QUERY: {user_query}
```

7. Citation

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, 12, 157–173. arXiv:2307.03172.